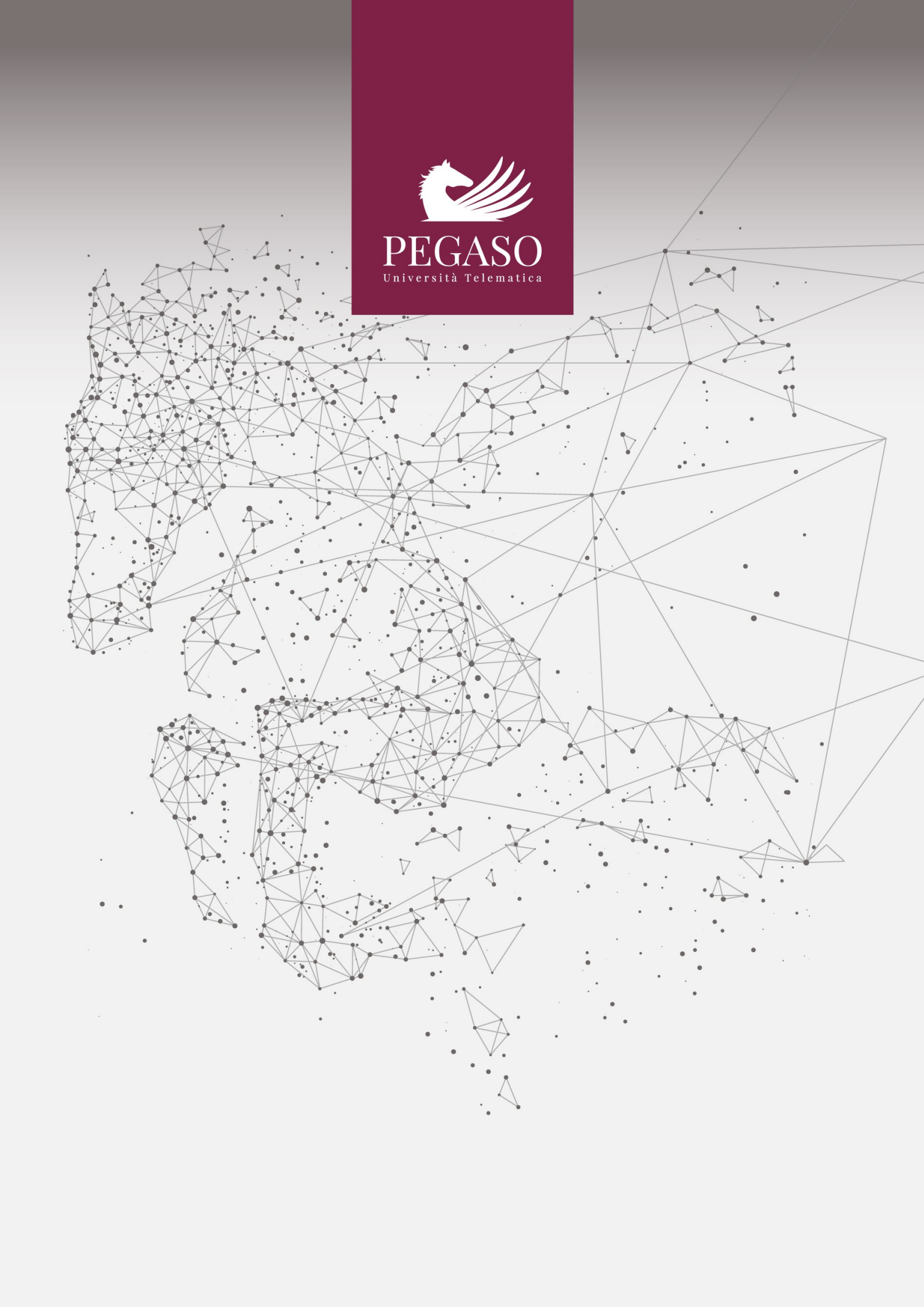

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. COMPLICAZIONI NEL DESIGN 3

2. STATEFUL VS STATELESS 6

3. OBJECT SERVER 8

BIBLIOGRAFIA 12

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Complicazioni nel design

Un server è un processo che implementa un servizio specifico per un gruppo di client. In sostanza, ogni server è strutturato in modo identico: attende una richiesta in arrivo da un client, si occupa di essa e quindi attende la successiva richiesta in arrivo.

Esistono numerosi metodi per organizzare i server. Nel caso di un server iterativo, il server gestisce la richiesta e restituisce una risposta al client richiedente, se necessario. Dopo aver passato la richiesta ad un thread o processo separato, un server concorrente attende immediatamente la successiva richiesta in ingresso. Un server con più thread è un esempio di server concorrente. Un metodo alternativo per implementare un server concorrente consiste nel fare un fork di un nuovo processo per ogni nuova richiesta. Questa metodologia è utilizzata da numerosi sistemi Unix. Il thread o il processo di gestione delle richieste è responsabile della restituzione di una risposta al client richiedente.

Un altro problema è la posizione da cui i client si connettono a un server. In tutti i casi, i client inviano richieste a un endpoint, noto anche come porta, sul computer host del server. Ogni server è sintonizzato su un particolare endpoint. Resta il problema di definire come i client determinano la conclusione di un servizio. Una strategia consiste nell'assegnare endpoint a livello globale per servizi noti. I server che gestiscono le richieste FTP, ad esempio, sono sempre in ascolto sulla porta TCP 21. Allo stesso modo, un server HTTP per il World Wide Web utilizzerà sempre la porta TCP 80 per l'ascolto. La Internet Assigned Numbers Authority (IANA) ha assegnato questi valori standard per tali endpoint. Con gli endpoint assegnati, il client deve solo determinare l'indirizzo di rete della macchina host del server. I servizi di naming possono svolgere questa funzione. Numerosi servizi non richiedono un endpoint predeterminato, come ad esempio un server time-of-the-day può utilizzare un endpoint assegnato dinamicamente dal suo sistema operativo locale. In questo caso, un client deve prima cercare l'endpoint. Una soluzione è installare un demone speciale su ogni macchina che esegue il server. Il demone monitora l'endpoint corrente di ogni servizio implementato da un server co-locato. Il demone stesso ascolta un endpoint riconoscibile. Come mostrato nella Figura 1, un client contatterà prima il demone, richiederà l'endpoint e quindi il server specifico. È comune collegare un endpoint a un particolare servizio.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

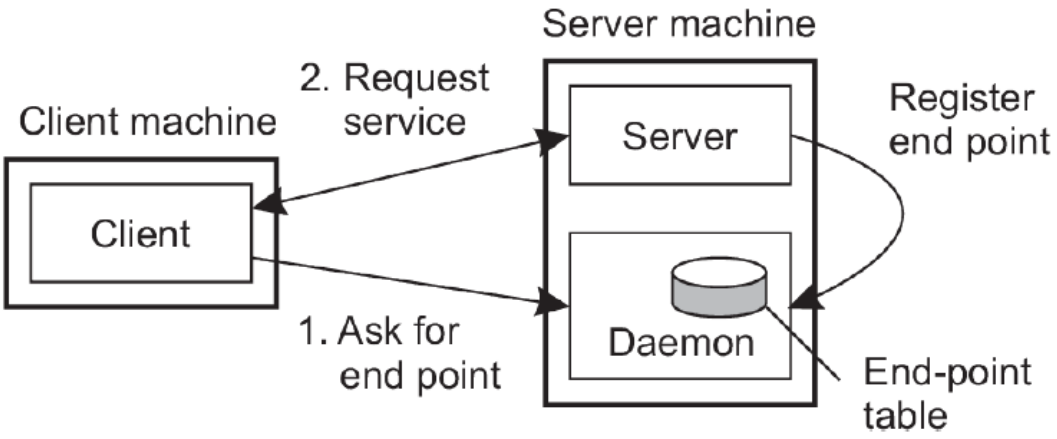
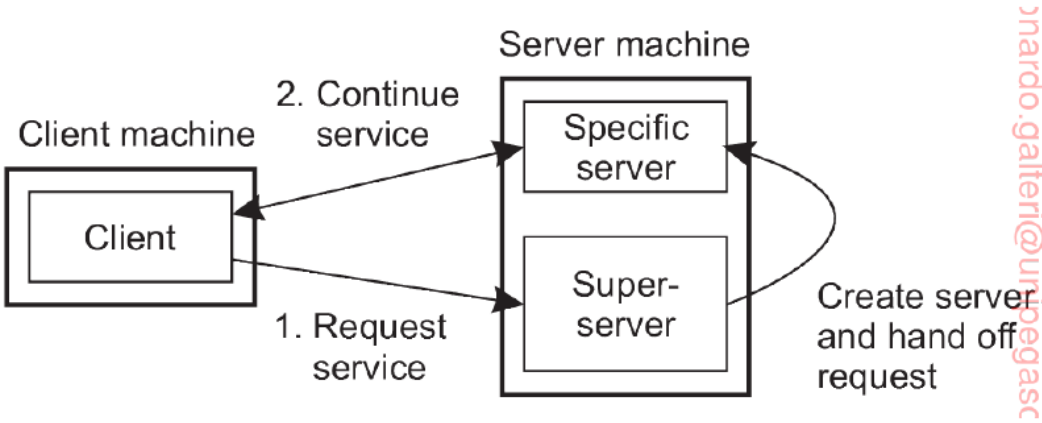


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Tuttavia, l'implementazione di ogni servizio su un server separato potrebbe rappresentare un utilizzo inefficiente delle risorse. In un tipico sistema Unix, ad esempio, è comune avere più server in esecuzione contemporaneamente, con la maggior parte di essi in attesa passiva di una richiesta client. Piuttosto che tenere traccia di così tanti processi passivi, è spesso più efficiente avere un singolo superserver in ascolto di ogni endpoint associato a un particolare servizio, come illustrato nella Figura 2. Il demone *inetd* in Unix, per esempio, ascolta su un certo numero di porte ben note per i servizi Internet. Quando una richiesta viene ricevuta, il demone fa un fork di un processo per gestirla.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



onardo.galteri@unipegasc

Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Quando si progetta un server, è anche necessario considerare se e come il server può essere interrotto. Si consideri, ad esempio, un utente che ha deciso di caricare un file enorme su un server FTP. Quindi, dopo essersi reso conto che si tratta del file errato, desidera interrompere la trasmissione dei dati interrompendo il server. Ci sono molti modi per raggiungere questo obiettivo. L'utente deve uscire bruscamente dall'applicazione client (che interromperà automaticamente la connessione al server), riavviarla e quindi agire come se nulla fosse accaduto. Il server terminerà infine la connessione precedente, supponendo che il client si sia arrestato in modo anomalo. Il modo migliore per gestire le interruzioni di comunicazione consiste nello sviluppare il client e il server in modo tale che i dati fuori banda possano essere inviati dal client al server prima di qualsiasi altro dato proveniente da tale client. Una soluzione consiste nell'ascoltare un endpoint di controllo separato a cui il client invia dati fuori banda, ascoltando contemporaneamente (con una priorità inferiore) l'endpoint attraverso il quale passano i dati normali. L'invio di dati fuori banda sulla stessa connessione attraverso la quale il client ha inviato la richiesta iniziale è un'altra opzione. Ad esempio, TCP consente la trasmissione di dati urgenti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Stateful vs stateless

Una considerazione cruciale per la progettazione è se il server è senza stato o stateless. Un server stateless non tiene traccia dello stato dei suoi client e può cambiare il proprio stato senza avvisare alcun client. Ad esempio, un server Web è stateless. Risponde semplicemente alle richieste HTTP in entrata, che possono essere per caricare un file sul server o (più frequentemente) per recuperare un file. Dopo l'elaborazione della richiesta, il server Web dimentica completamente il client. Analogamente, l'insieme di file gestiti da un server Web (eventualmente in combinazione con un file server) può essere modificato senza avvisare i client.

Si noti che in molte progettazioni stateless, il server mantiene informazioni sui propri client; tuttavia, se queste informazioni vengono perse, il servizio del server non verrà interrotto. In genere, un server Web registra tutte le richieste client. Queste informazioni sono utili per determinare, ad esempio, se e dove devono essere replicati documenti specifici. Chiaramente, se il log viene perso, non si hanno penalità se non prestazioni eventualmente non ottimali.

Un tipo specifico di architettura senza stato è quello in cui il server mantiene quello che è noto come stato soft. In questo caso, il server promette di mantenere lo stato per conto del client, ma solo per un periodo di tempo limitato. Trascorso questo periodo di tempo, il server ripristina il comportamento predefinito ed elimina tutte le informazioni specifiche del client. Questo tipo di stato si può riscontrare nel caso in cui un server è impegnato a tenere informato un client sugli aggiornamenti per un periodo di tempo limitato. Gli approcci soft-state derivano dalla progettazione di protocolli nelle reti di computer, ma possono essere applicati anche alla progettazione di server.

Un server con stato o stateful, d'altra parte, in genere memorizza informazioni persistenti sui propri client. Ciò significa che il server deve eliminare le informazioni in modo esplicito. Un file server che consente a un client di conservare una copia locale di un file, anche per le operazioni di aggiornamento, è un esempio tipico. Tale server manterrebbe una tabella con le voci per (client, file). Tale tabella consente al server di tenere traccia di quale client dispone attualmente delle autorizzazioni di aggiornamento su quale file e quindi potenzialmente della versione più aggiornata di tale file.

Questa strategia può migliorare le prestazioni percepite delle operazioni di lettura e scrittura da parte del client. Le prestazioni migliorate dei server stateful rispetto ai server senza stato sono un vantaggio comune di tali progettazioni. Tuttavia, se il server si arresta in modo anomalo, deve recuperare la tabella delle voci (client, file); in caso contrario, non può garantire che gli aggiornamenti più recenti di un file siano stati elaborati. In genere, un server stateful deve ripristinare l'intero stato esistente prima dell'arresto

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

anomalo. Consentire il recupero può introdurre una complessità significativa. In un'architettura senza stato, un server bloccato può essere ripristinato senza passaggi aggiuntivi. Riprende semplicemente il funzionamento e attende l'arrivo delle richieste dei clienti.

Dovrebbe essere fatta una distinzione tra stato di sessione (temporaneo) e stato permanente. L'esempio precedente è tipico dello stato di sessione: esso è associato a una serie di operazioni eseguite da un singolo utente e deve essere mantenuto per un certo periodo di tempo, ma non per sempre. Lo stato di sessione viene in genere mantenuto in architetture client-server a tre livelli, in cui il server delle applicazioni deve eseguire una serie di query sul server database prima di rispondere al client richiedente. Il problema in questione è che se lo stato di sessione viene perso, non vi è alcun danno reale finché il client può semplicemente riemettere la richiesta originale. Questa osservazione consente un'archiviazione dello stato meno affidabile e più semplice.

In genere, le informazioni mantenute nei database, ad esempio le informazioni sui clienti, le chiavi di licenza software e così via, rimangono se lo stato è permanente. Tuttavia, per la maggior parte dei sistemi distribuiti, il mantenimento dello stato di sessione implica già una progettazione stateful che richiede misure speciali quando si verificano errori e presupposti espliciti relativi alla durabilità dello stato memorizzato sul server.

Quando si progetta un server, la scelta tra una progettazione senza stato e una progettazione con stato non deve influire sui servizi offerti dal server. Ad esempio, se i file devono essere aperti prima di poter essere letti o scritti, un server senza stato dovrebbe imitare questo comportamento in qualche modo. In genere, un server risponde a una richiesta di lettura o scrittura aprendo il file di riferimento, eseguendo l'effettiva operazione di lettura o scrittura e quindi chiudendo immediatamente il file.

In altri casi, un server potrebbe voler tenere traccia del comportamento di un client per rispondere alle sue richieste in modo più efficace. I server Web, ad esempio, occasionalmente offrono la possibilità di indirizzare immediatamente un utente alle sue pagine preferite. Questo metodo è possibile solo se il server dispone di informazioni sulla cronologia specifiche del client. Quando un server non è in grado di mantenere lo stato, una soluzione comune è che il client invii informazioni aggiuntive relative ai suoi accessi precedenti. Nel caso del World Wide Web, queste informazioni vengono in genere memorizzate in modo trasparente dal browser del cliente in un cookie, che è un piccolo pezzo di dati contenente informazioni specifiche del cliente che sono di interesse per il server. Un browser non esegue mai i cookie; sono semplicemente memorizzati. La prima volta che un client si connette a un server, il server invia un cookie insieme alle pagine Web richieste al browser, che quindi memorizza il cookie. Ogni volta che il client accede successivamente al server, il cookie viene trasmesso insieme alla richiesta.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Object server

Esaminiamo la struttura generale degli object server necessari per gli oggetti distribuiti. Una distinzione importante tra un object server e altri server (più convenzionali) è che un object server non fornisce un servizio specifico da solo. Gli oggetti che risiedono sul server implementano servizi particolari. Il server fornisce essenzialmente solo i mezzi per richiamare oggetti locali in risposta alle richieste dei client remoti. Di conseguenza, è relativamente semplice modificare i servizi aggiungendo e rimuovendo oggetti.

Di conseguenza, un object server funge da residenza per gli oggetti. Due componenti costituiscono un oggetto: i dati che rappresentano il suo stato e il codice per l'esecuzione dei suoi metodi. Il fatto che questi componenti siano separati o che più oggetti condividano le implementazioni dei metodi dipende dall'object server. Esistono anche variazioni nel modo in cui un object server richiama i propri oggetti. In un server multithreading, a ogni oggetto può essere assegnato un thread separato oppure ogni richiesta di chiamata può utilizzare un thread separato.

Affinché un oggetto venga richiamato, un object server deve sapere quale codice eseguire, su quali dati operare, se è necessario creare un thread separato per gestire la chiamata e così via. Una strategia semplice consiste nell'assumere che tutti gli oggetti abbiano lo stesso aspetto e che esista un solo modo per richiamare un oggetto. Sfortunatamente, tale approccio è in genere poco flessibile e spesso limita inutilmente gli sviluppatori di oggetti distribuiti. Un server che supporta più criteri è un metodo di gran lunga superiore.

Si consideri, ad esempio, un oggetto transitorio, che esiste solo per il tempo di attività del suo server, o possibilmente per una durata più breve. Una copia di sola lettura di un file archiviato in memoria può in genere essere implementata come oggetto temporaneo. Una calcolatrice potrebbe essere implementata in modo simile come oggetto transitorio. Un criterio ragionevole consiste nel creare un oggetto temporaneo alla prima richiesta di chiamata e distruggerlo quando non è più associato ad alcun client.

Questo metodo presenta il vantaggio che un oggetto temporaneo utilizzerà le risorse del server solo per il tempo effettivamente necessario. Lo svantaggio è che una chiamata può richiedere del tempo per essere completata, poiché l'oggetto deve prima essere creato. Pertanto, un criterio alternativo consiste nel creare tutti gli oggetti temporanei al momento dell'inizializzazione del server, a scapito dell'utilizzo di risorse quando nessun client utilizza l'oggetto.

Analogamente, un server potrebbe implementare il criterio secondo cui ciascuno dei suoi oggetti risiede nel proprio segmento di memoria. In altre parole, né il codice né i dati sono condivisi tra gli oggetti.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Quando un'implementazione di un oggetto non separa codice e dati o quando gli oggetti devono essere separati per motivi di sicurezza, questo criterio può essere necessario. In quest'ultimo scenario, il server deve prendere precauzioni speciali o richiedere assistenza al sistema operativo sottostante per evitare violazioni dei limiti del segmento.

La strategia alternativa consiste nel consentire agli oggetti di condividere almeno il codice. Ad esempio, un database contenente oggetti della stessa classe può essere implementato in modo efficiente caricando l'implementazione della classe nel server una sola volta. Quando viene ricevuta una richiesta di chiamata di un oggetto, il server deve solo recuperare lo stato dell'oggetto ed eseguire il metodo richiesto.

Allo stesso modo, ci sono una varietà di criteri relativi al threading. L'utilizzo di un singolo thread di controllo per implementare il server è il metodo più semplice. Il server può anche utilizzare più thread, uno per ciascuno dei suoi oggetti. Ogni volta che il server riceve una richiesta di chiamata per un oggetto, inoltra la richiesta al thread responsabile di tale oggetto. La richiesta viene temporaneamente accodata se il thread è attualmente occupato.

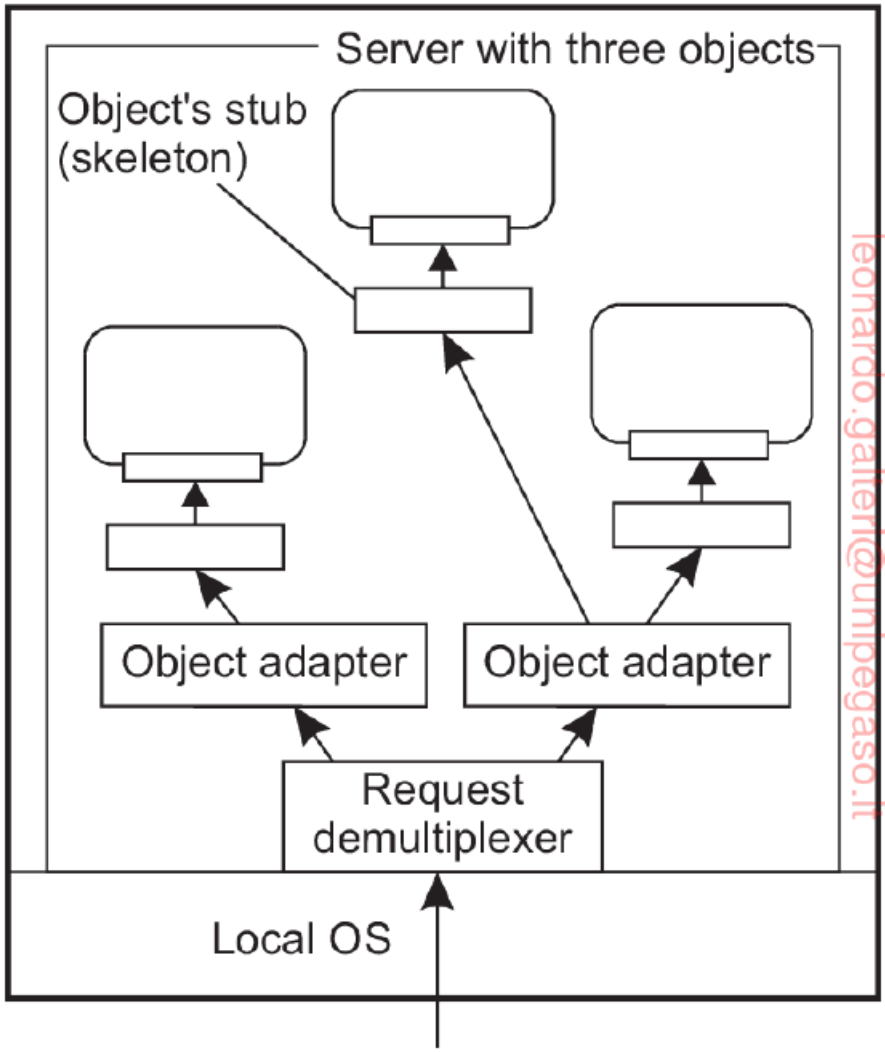
Il vantaggio di questo metodo è che gli oggetti sono automaticamente protetti dall'accesso simultaneo: tutte le chiamate vengono serializzate tramite il thread dell'oggetto. Ovviamente, è anche possibile utilizzare un thread separato per ogni richiesta di chiamata, sebbene la protezione dell'accesso simultaneo debba già esistere per gli oggetti. Il fatto che i thread vengano creati su richiesta o che il server gestisca un pool di thread è indipendente dal fatto che venga creato un thread per oggetto o per metodo. In genere non esiste un'unica politica ottimale. La scelta dipende dalla disponibilità dei thread, dall'importanza delle prestazioni e da altri fattori.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Comunemente, le decisioni relative a come richiamare un oggetto sono indicate come criteri di attivazione, per sottolineare che in molti casi, l'oggetto stesso deve essere portato nello spazio degli indirizzi del server (cioè attivato) prima di essere invocato. Quindi, è necessario un meccanismo per raggruppare gli oggetti in base ai criteri. Questo meccanismo viene talvolta definito adattatore di oggetti o wrapper di oggetti. Un adattatore oggetti è meglio inteso come software che implementa un particolare criterio di attivazione. Il problema principale, tuttavia, è che gli adattatori dell'oggetto vengono forniti come componenti generici per assistere gli sviluppatori di oggetti distribuiti e richiedono solo la configurazione specifica dei criteri.

Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition



leonardo.galteri@unipegaso.it

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un adattatore di oggetti ha il controllo su uno o più oggetti. Più adattatori di oggetti possono risiedere sullo stesso server, poiché un server deve essere in grado di supportare contemporaneamente oggetti con criteri di attivazione diversi. Come illustrato nella Figura 3, quando una richiesta di chiamata viene recapitata al server, la richiesta viene prima inviata all'adattatore di oggetti appropriato.

È importante sottolineare che gli adattatori di oggetti sono ignari delle interfacce specifiche degli oggetti che controllano. Non potrebbero mai essere generici altrimenti. L'unica preoccupazione di un adattatore di oggetti è che può estrarre un riferimento a un oggetto da una richiesta di chiamata e quindi inviare la richiesta all'oggetto a cui si fa riferimento in base a un criterio di attivazione specifico. Anziché passare la richiesta direttamente all'oggetto, come illustrato nella Figura 3, un adattatore invia una richiesta di chiamata allo stub sul lato server di tale oggetto. Il server stub, noto anche come scheletro, viene in genere generato dalle definizioni dell'interfaccia dell'oggetto, esegue l'operazione di unmarshalling della richiesta e chiama il metodo appropriato.

Configurandolo in fase di esecuzione, un adattatore oggetti può supportare più criteri di attivazione. Nei sistemi conformi a CORBA, ad esempio, è possibile specificare se un oggetto deve continuare ad esistere dopo che l'adattatore associato ha cessato di funzionare. Analogamente, è possibile configurare un adattatore per generare identificatori di oggetto o consentire all'applicazione di fornirne uno. Come descritto in precedenza, è possibile configurare una scheda per funzionare in modalità a thread singolo o multithread.

Si noti che, sebbene siano stati discussi gli oggetti nella Figura 3, non è stato specificato quali siano questi oggetti. In particolare, va sottolineato che il server può (indirettamente) accedere a database o chiamare routine di libreria speciali come parte dell'implementazione di tale oggetto. Gli adattatori di oggetto che comunicano solo con gli scheletri sono protetti dai dettagli di implementazione. Di conseguenza, l'implementazione effettiva potrebbe non avere nulla in comune con gli oggetti a livello di linguaggio (cioè in fase di compilazione) tipicamente osservati. Di conseguenza, viene tipicamente utilizzata una terminologia distinta. Un servant è il termine generale per un pezzo di codice che costituisce l'implementazione di un oggetto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Reynolds J. and Postel J. Assigned Numbers. RFC 1700, 1994.
- Birman K. Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services, 2012.
- Clark D. The Design Philosophy of the DARPA Internet Protocols. In SIGCOMM, 1989.
- Lui J., Misra V., and Rubenstein D. On the Robustness of Soft State Protocols. In 12th International Conference on Network Protocols, 2004.
- B. C., Kiciman E., and Fox A. Session State: Beyond Soft State. In 1st Symposium on Networked Systems Design and Implementation, 2004.
- OMG. The Common Object Request Broker: Architecture and Specification, revision 2.4.2. OMG Document formal/00-02-33, Object Management Group, 2001.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).